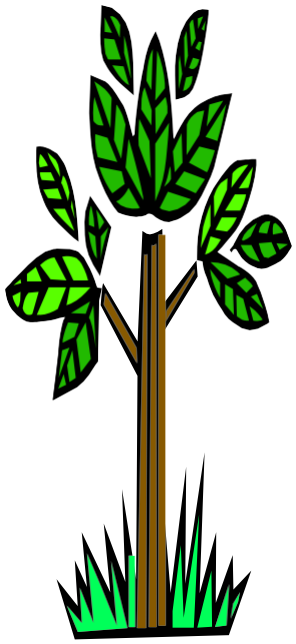
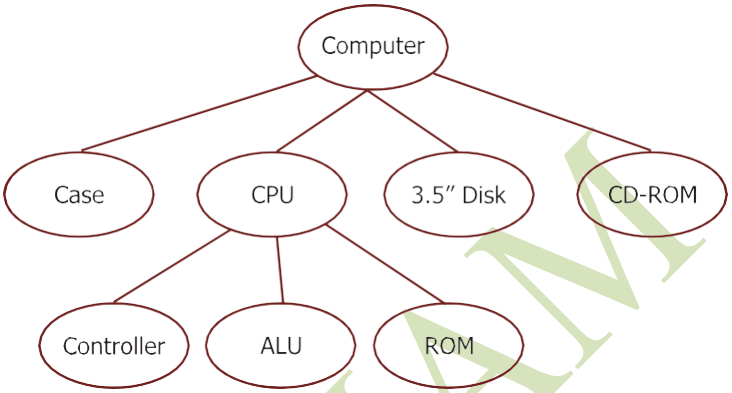
BÀI 5: CÂY





Mục tiêu
Sau khi học bài này, các bạn có thể: • Mô tả các khái niệm về cây, cây nhị phân, cây tổng quát một cách chính xác. • Trình bày các cách cài đặt về cây tổng quát, cây nhị phân và thực hiện cài đặt các thao tác trên cây nhị phân một cách chính xác bằng một ngôn ngữ lập trình C. • Xác định đúng một số ứng dụng của cây nhị phân và cây trò chơi. • Sử dụng cấu trúc dữ liệu dạng cây cho phù hợp để giải quyết một số bài toán.

Nội dung
• Các khái niệm cơ bản về cây. • Cây nhị phân. • Ứng dụng của cây nhị phân. • Cây tổng quát. • Cây biểu diễn trò chơi.

Thời lượng học
10 tiết

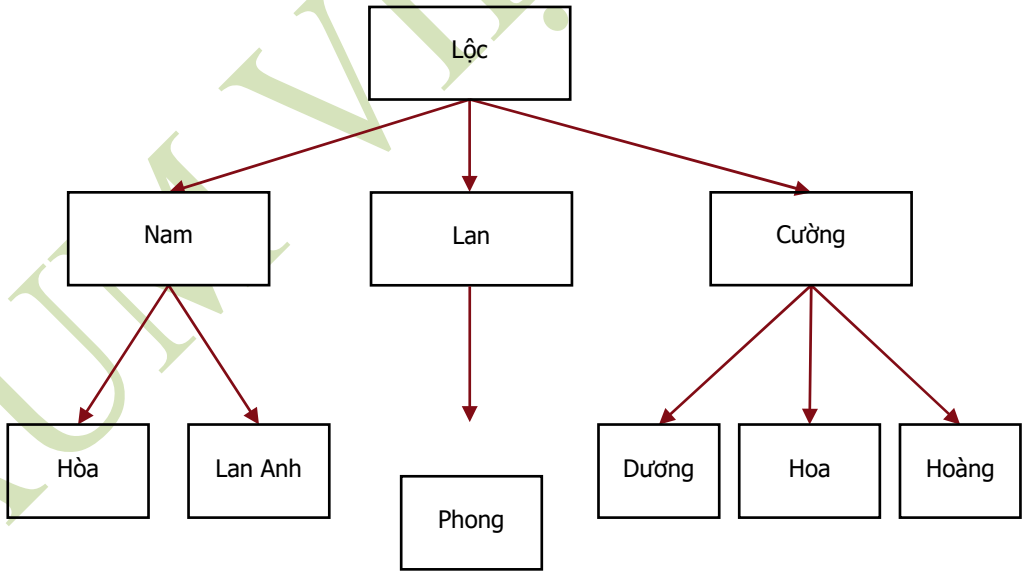
ADMVIETNAM

Trong bài này chúng ta sẽ nghiên cứu mô hình dữ liệu cây. Cây là một cấu trúc dữ liệu rất phổ biến trong khoa học máy tính, là một cấu trúc dữ liệu phi tuyến tính và được sử dụng để lưu dữ liệu theo một thứ tự nào đó. Cây được sử dụng rộng rãi trong rất nhiều vấn đề khác nhau. Chẳng hạn, nó được áp dụng để tổ chức thông tin trong các hệ cơ sở dữ liệu, để mô tả cấu trúc cú pháp của các chương trình nguồn khi xây dựng các chương trình dịch. Rất nhiều các bài toán trong các lĩnh vực khác nhau được quy về việc thực hiện các phép toán trên cây. Do vậy, trong bài này chúng ta sẽ tìm hiểu định nghĩa và các khái niệm cơ bản về cây. Chúng ta cũng sẽ xét các phương pháp cài đặt cây và sự thực hiện các phép toán cơ bản trên cây. Sau đó chúng ta sẽ nghiên cứu kỹ một dạng cây đặc biệt, đó là cây nhị phân.

5.1. Các khái niệm cơ bản về cây

Cây là một cấu trúc khá quen thuộc và rất thuận tiện cho việc biểu diễn những kết cấu phân cấp dạng rẽ nhánh. Nó liên tưởng đến cây trong đời sống thực và cũng thường dùng các thuật ngữ liên quan như: gốc, cành, lá. Đặc điểm cơ bản của cấu trúc này là các nút có thứ bậc. Một nút cha có một hoặc nhiều nút con. Mỗi nút con chỉ trực thuộc một nút cha. Các nút con trực thuộc 1 nút cha có quan hệ anh em... Các nút sau cùng (không có con) gọi là các nút lá. Số lượng cành từ một nút đến gốc chính là bậc của nút đó. Các nút có cùng thứ bậc tạo thành 1 lớp.

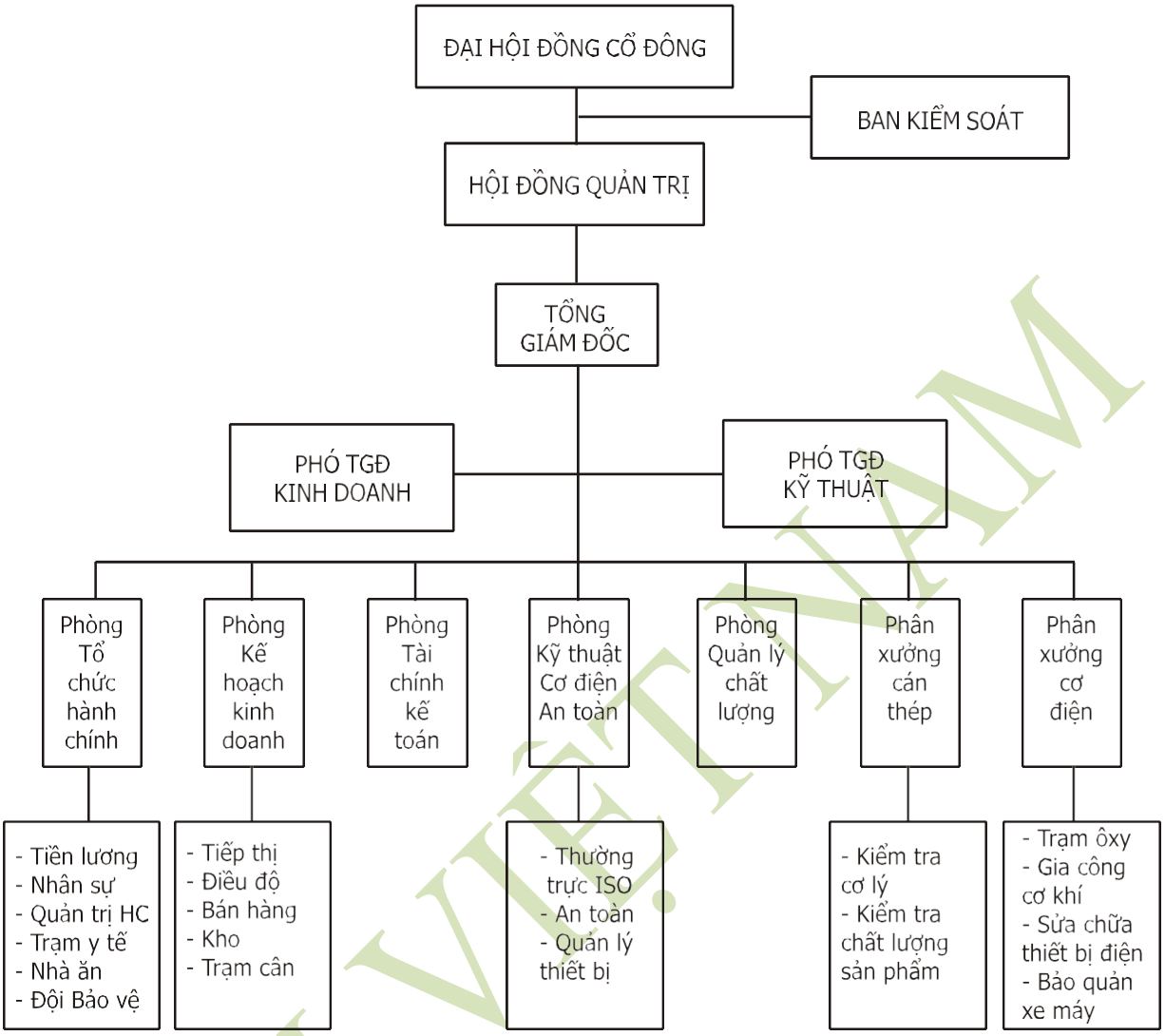
Trong cuộc sống, chúng ta vẫn thường xuyên sử dụng sơ đồ dạng cây để biểu diễn mối quan hệ giữa các thành viên trong một gia đình, một dòng họ hay biểu diễn mô hình tổ chức của một công ty, một tổ chức xã hội nào đó… Bây giờ ta hay quan sát sơ đồ quan hệ của một gia đình dưới đây:



Từ sơ đồ trên ta thấy được mối quan hệ giữa các thành viên trong gia đình: Lộc có ba người con là Nam, Lan và Cường, Hòa và Lan Anh là hai anh em ruột, Hòa là cháu họ của Lan và Cường, Phong là cháu ngoại của Lộc, Phong và Hoàng là anh em họ với nhau. Như vậy từ sơ đồ trên ta dễ dàng xác định được mối quan hệ giữa các thành viên trong gia đình.

Các tổ chức xã hội cũng thường có cấu trúc cây và thường được mô tả bằng 1 sơ đồ cây.

AUM VIỆT NAM



Trong khoa học máy tính, để biểu diễn mối quan hệ của các đối tượng trên các sơ đồ này, người ta đã xây dựng một cấu trúc dữ liệu gọi là cấu trúc dữ liệu dạng cây – gọi tắt là **Cây (Tree).**

Định nghĩa

Cây là một cấu trúc dữ liệu trừu tượng gồm một tập hữu hạn các phần tử gọi là nút (hay đỉnh), giữa các nút có một quan hệ phân cấp gọi là quan hệ "cha – con" và một tập hợp hữu hạn những cạnh nối các cặp nút cha – con với nhau. Nếu cây không rỗng, có một nút gọi là nút gốc (root).

Có thể định nghĩa cây bằng các đệ quy như sau:

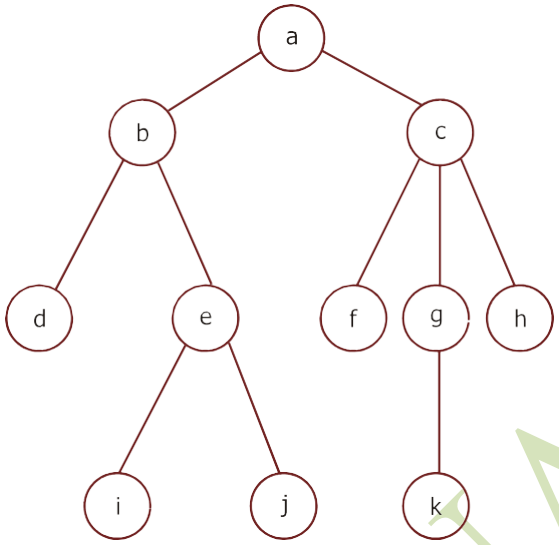
Mỗi nút là một cây, nút đó cũng là gốc của cây ấy.

Nếu n là 1 nút và $T_1, T_2,\ldots, T_k$ là các cây với $n_1, n_2,\ldots, n_k$ lần lượt là các gốc, thì 1 cây mới T sẽ được tạo lập bằng cách cho n trở thành cha của các nút $n_1, n_2,\ldots, n_k$; nghĩa là trên cây này n là gốc còn $T_1, T_2,\ldots, T_k$ là các các cây con (subtrees) của gốc. Lúc đó, $n_1, n_2,\ldots, n_k$ là con của nút n.

Để tiện, người ta cho phép tồn tại 1 cây không có nút nào gọi là cây rỗng (null tree).

ATM VIỆT NAM

Ví dụ 5.1. Xét một cây có dạng như sau:



Hình 5.1. ***Cây***

Hình 5.1 minh hoạ một cây T. Đó là một tập hợp T gồm 11 phần tử, T = {a, b, c, d, e, f, g, h, i, j, k}. Các phần tử của T được gọi là các đỉnh của cây T. Tập T có cấu trúc như sau. Các đỉnh của T được phân thành các lớp không cắt nhau: lớp thứ nhất gồm một đỉnh duy nhất a, đỉnh này gọi là *gốc* của cây; lớp thứ hai gồm các đỉnh b, c ; lớp thứ ba gồm các đỉnh d, e, f, g, h và lớp cuối cùng gồm các đỉnh i, j, k, mỗi đỉnh thuộc một lớp (trừ gốc), có một cung duy nhất nối với một đỉnh nào đó thuộc lớp kề trên. (Cung này biểu diễn mối quan hệ nào đó).

Một số khái niệm cơ bản về cây

Từ định nghĩa về cây ta thấy rằng, mỗi đỉnh của cây là gốc của các cây con của nó. Số các cây con của một đỉnh gọi là **bậc** của đỉnh đó. Các đỉnh có bậc bằng không được gọi là **lá** của cây.

Nếu đỉnh b là gốc của một cây con của đỉnh a thì ta nói đỉnh b là con của đỉnh a và a là cha của b. Như vậy, **bậc** của một đỉnh là số các đỉnh con của nó, còn lá là đỉnh không có con. Các đỉnh có ít nhất một con được gọi là **đỉnh trong**. Các đỉnh của cây hoặc là lá hoặc là đỉnh trong. Các đỉnh có cùng một cha được gọi là anh em.

Một dãy các đỉnh $a_1, a_2, \ldots, a_n$ ($n \geq 1$), sao cho a_i ($i = 1, 2, \ldots, n-1$) là cha của a_{i+1} được gọi là đường đi từ a_1 đến a_n. **Chiều dài đường đi** này là số cành trên nó đó là $n - 1$. Ta có nhận xét rằng, luôn luôn tồn tại một đường đi duy nhất từ gốc tới một đỉnh bất kỳ trong cây. Mỗi nút có đường đi với chiều dài bằng 0 đến chính nó.

Nếu có một đường đi từ đỉnh a đến đỉnh b có độ dài $k \geq 1$, thì ta nói a là tiền thân của b và b là hậu thế của a.

Ví dụ 5.2.

Trong cây ở hình 5.1, đỉnh c là cha của đỉnh f, g, h. Các đỉnh d, i, j, k, f và h là lá, các đỉnh còn lại là đỉnh trong. a, c, g, k là đường đi có độ dài 3 từ a đến k. Đỉnh b là tiền thân của các đỉnh d, e, i, j.

Cây con.

Từ định nghĩa cây ta có, mỗi đỉnh bất kỳ của cây T là gốc của một cây nào đó, ta gọi cây này là cây con của cây T. Nó gồm đỉnh a và tất cả các đỉnh còn lại là hậu thế của a. Chẳng hạn, với cây T trong hình 5.1, T_1 = {c, f, g, h, k} là một cây con.

AUM VIỆT NAM

Độ cao, mức của 1 đỉnh.

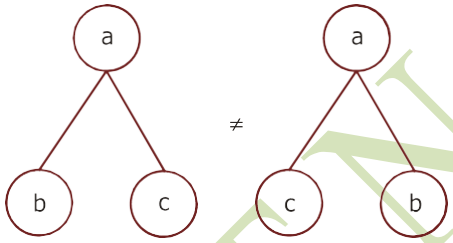
Trong một cây, **độ cao** của một đỉnh a là độ dài của đường đi dài nhất từ a đến các lá của nó. Độ cao của gốc được gọi là độ cao của cây. **Mức (độ sâu)** của đỉnh a là độ dài của đường đi từ gốc đến a. Như vậy gốc có mức 0.

Ví dụ 5.3.

Trong cây ở hình 5.1, đỉnh b có độ cao là 2, cây có độ cao là 3. Các đỉnh b, c có mức 1; các đỉnh d, e, f, g, h có mức 2, còn mức của các đỉnh i, j, k là 3.

Cây được sắp.

Trong một cây, nếu các cây con của mỗi đỉnh được sắp theo một thứ tự nhất định, thì cây được gọi là cây được sắp. Chẳng hạn, hình 4.3 minh hoạ hai cây được sắp khác nhau.



Hình 5.2. ***Hai cây được sắp khác nhau***

Giả sử trong một cây được sắp T, đỉnh a có các con được sắp theo thứ tự: b_1, b_2,…, b_k ($k \geq 1$). Khi đó ta nói, b_1 là con trưởng của a, và b_i là anh liền kề của b_{i+1} (b_{i+1} là em liền kề của b_i), i = 1, 2, ..., k – 1. Ta còn nói, với i < j thì b_i ở bên trái b_j (b_j ở bên phải b_i). Quan hệ này được mở rộng như sau. Nếu a ở bên trái b thì mọi hậu thế của a ở bên trái mọi hậu thế của b.

Ví dụ: trong hình 5.1, f là con trưởng của c, và là anh liền kề của đỉnh g. Đỉnh i ở bên trái đỉnh g.

Cây gắn nhãn.

Cây gắn nhãn là cây mà mỗi đỉnh của nó được gắn với một giá trị (nhãn) nào đó. Nói một cách khác, cây gắn nhãn là một cây cùng với một ánh xạ từ tập hợp các đỉnh của cây vào tập hợp nào đó các giá trị (các nhãn). Chúng ta có thể xem nhãn như thông tin liên kết với mỗi đỉnh của cây. Nhãn có thể là các dữ liệu đơn như số nguyên, số thực, hoặc cũng có thể là các dữ liệu phức tạp như bản ghi. Cần biết rằng, các đỉnh khác nhau của cây có thể có cùng một nhãn.

Rừng.

Một rừng F là một danh sách các cây:

$$F = (T_1, T_2, \ldots, T_n)$$

trong đó T_i (i = 1, ..., n) là cây (cây được sắp).

Chúng ta có tương ứng một – một giữa tập hợp các cây và tập hợp các rừng. Thật vậy, một cây T với gốc r và các cây con của gốc theo thứ tự từ trái sang phải là T_1, T_2,…, T_n, $T = (r, T_1, T_2, \ldots, T_n)$ tương ứng với rừng $F = (T_1, T_2, \ldots, T_n)$ và ngược lại.

ACM VIỆT NAM

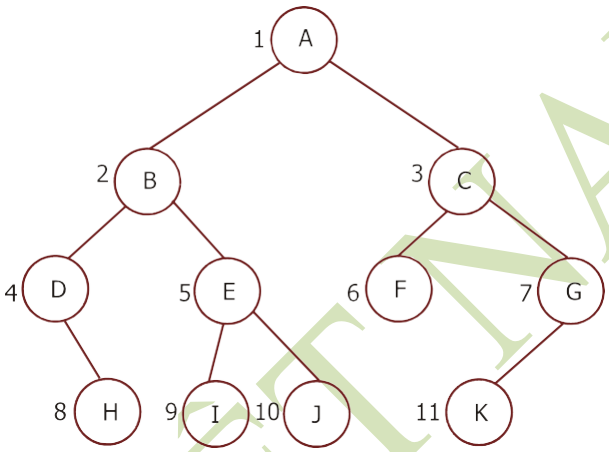
5.2. Cây nhị phân

5.2.1. Khái niệm

Khái niệm

Một trong những dạng quan trọng của cấu trúc cây đó là cây nhị phân. Cây nhị phân là một cây mà trong đó mỗi nút có nhiều nhất hai con.

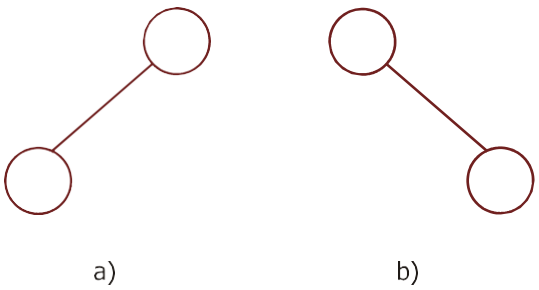
Từ định nghĩa cây nhị phân, ta suy ra rằng, mỗi đỉnh của cây nhị phân chỉ có nhiều nhất là hai đỉnh con, một đỉnh con bên trái (đó là gốc của cây con trái) và một đỉnh con bên phải (đó là gốc của cây con phải).



Hình 5.3. ***Một cây nhị phân***

Cần lưu ý rằng, cây (cây có gốc) và cây nhị phân là hai khái niệm khác nhau. Cây không bao giờ trống, nó luôn luôn chứa ít nhất một đỉnh, mỗi đỉnh có thể không có, có thể có một hay nhiều cây con. Còn cây nhị phân có thể trống, mỗi đỉnh của nó luôn luôn có hai cây con được phân biệt là cây con bên trái và cây con bên phải.

Chẳng hạn, hình 5.4 minh hoạ hai cây nhị phân khác nhau. Cây nhị phân trong hình 5.4a có cây con trái của gốc gồm một đỉnh, còn cây con phải trống. Cây nhị phân trong hình 5.4b có cây con trái của gốc trống, còn cây con phải gồm một đỉnh. Song ở đây ta chỉ có một cây: đó là cây mà gốc của nó chỉ có một cây con gồm một đỉnh.

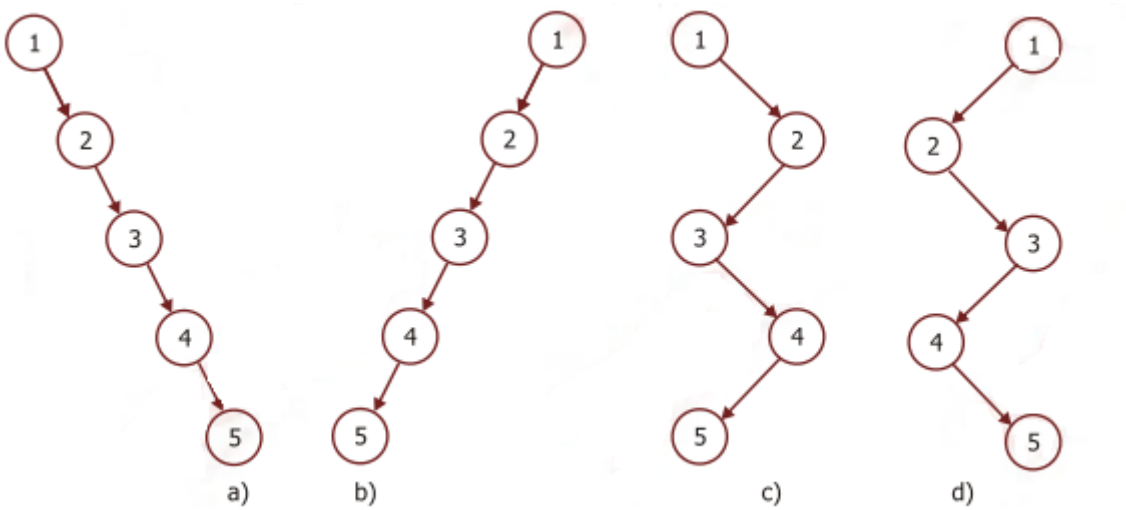


Hình 5.4. ***Hai cây nhị phân khác nhau***

Một số dạng đặc biệt của cây nhị phân:

- **Cây nhị phân lệch phải, cây nhị phân lệch trái, cây zíc-zắc**. Các dạng cây nhị phân này thường gọi là cây nhị phân suy biến.

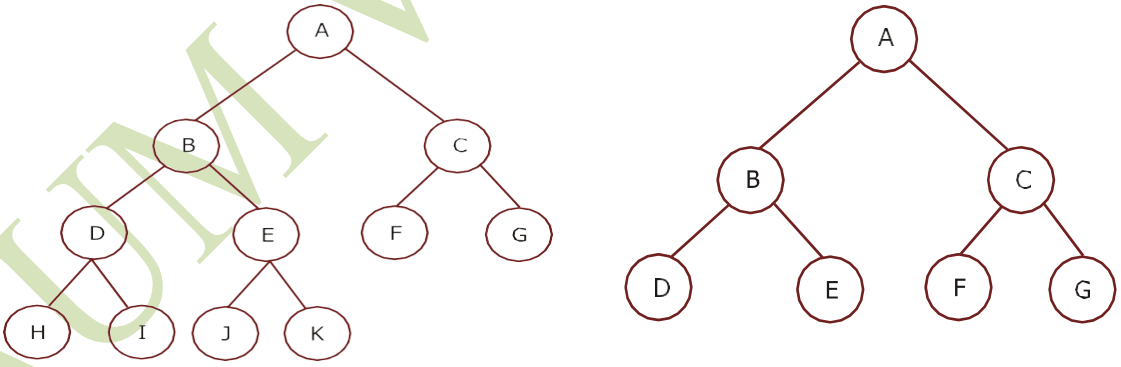
AUM VIỆT NAM



Hình 5.5. ***Các dạng cây nhị phân suy biến***

Trong hình 5.5, cây a gọi là cây lệch phải, cây b được gọi là cây lệch trái, các cây c và d gọi là cây zíc-zắc.

- **Cây nhị phân hoàn chỉnh (Complete binary tree):** cây nhị phân mà mọi nút có mức < h – 1 đều có đúng 2 nút con với h là chiều cao của cây (hay nói cách khác, các nút trong đều có 2 con).
- **Cây nhị phân đầy đủ (Full binary tree):** cây nhị phân mà mọi nút có mức ≤ h – 1 đều có đúng 2 nút con với h là chiều cao của cây (hay nói cách khác, cây nhị phân đầy đủ là cây nhị phân hoàn chỉnh mà tất cả các cây con của nút gốc đều có độ cao như nhau).



Hình 5.6. ***Cây nhị phân hoàn chỉnh (a) và cây nhị phân đầy đủ (b)***

a) b)

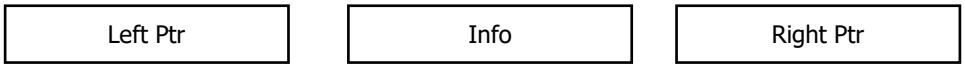
5.2.2. Cài đặt cây nhị phân

Cách thông dụng nhất là ta sử dụng cấu trúc liên kết để biểu diễn cây nhị phân. Khi đó, mỗi nút của cây là một bản ghi gồm 3 trường:

- **Trường Info**: chứa giá trị lưu tại nút đó.
- **Trường Left Ptr:** Chứa liên kết (con trỏ) tới nút con trái, tức là chứa thông tin đủ để nhận biết nút con trái của nút đó là nút nào. Trong trường hợp không có nút con trái, trường này được gắn một giá trị đặc biệt là giá trị trống (null).

AUM VIỆT NA

- **Trường Right Ptr:** Chứa liên kết (con trỏ) tới nút con phải, tức là chứa thông tin đủ để nhận biết nút con phải của nút đó là nút nào, trong trường hợp không có nút con phải, trường này được gắn một giá trị đặc biệt (giá trị trống – null).



Hình 5.7. ***Cấu trúc liên kết biểu diễn một nút của cây nhị phân***

Dưới đây ta sẽ cài đặt cây nhị phân bằng cấu trúc bằng mảng và bằng con trỏ.

5.2.2.1. Cài đặt bằng mảng

```
#define max N;
typedef <ten_kieu_du_lieu> Item;
typedef struct Node
        {
            Item infor;
            int left;
            int right;
         };
Node Tree[N];
```

Ta sử dụng một mảng để lưu giữ các đỉnh của cây nhị phân. Mỗi đỉnh của cây được biểu diễn bởi bản ghi gồm ba trường: trường infor mô tả thông tin gắn với mỗi đỉnh, trường trái (left) chỉ đỉnh con trái, trường phải (right) chỉ đỉnh con phải. Giả sử các đỉnh của cây được đánh số theo thứ tự lần lượt từ mức 1 trở đi, hết mức này đến mức khác và từ trái sang phải đối với các nút ở mỗi mức. Khi đó, cấu trúc dữ liệu biểu diễn cây nhị phân được khai báo như sau:

Hình dưới đây minh họa cấu trúc dữ liệu biểu diễn cây nhị phân trong hình 5.3 bằng mảng

	infor	left	right
1	A	2	3
2	B	4	5
3	C	6	7
4	D	0	8
5	E	9	10
6	F	0	0
7	G	11	0
8	H	0	0
9	I	0	0
10	J	0	0
	K	0	0

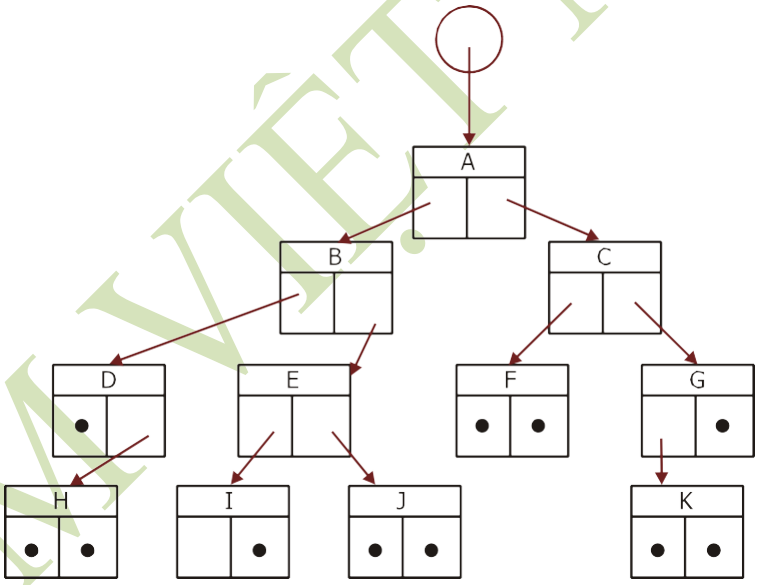
AUM VIỆT NAM

5.2.2.2. Cài đặt bằng con trỏ

Ngoài cách cài đặt cây nhị phân bởi mảng, chúng ta còn có thể sử dụng con trỏ để cài đặt cây nhị phân. Trong cách này mỗi bản ghi biểu diễn một đỉnh của cây chứa hai con trỏ: con trỏ trái (left) trỏ tới đỉnh con trái, con trỏ phải (right) trỏ tới đỉnh con phải. Ta có khai báo sau đây cho cây nhị phân:

```
typedef <kieu_du_lieu> ElementType;
//cấu trúc một nút
struct Node
        {
                ElementType infor;
                struct TreeNode *left;
                struct TreeNode *right;
        };
typedef struct Node *Tree;//định nghĩa một cây
Tree Root;//con trỏ trỏ tới nút gốc của cây
```

Với cách cài đặt này, cấu trúc dữ liệu biểu diễn cây nhị phân trong hình 5.3 được minh hoạ bởi hình 5.9.



Hình 5.9. ***Cấu trúc dữ liệu biểu diễn cây***

Từ nay về sau chúng ta sẽ chỉ sử dụng cách biểu diễn bằng con trỏ của cây nhị phân. Các phép toán đối với cây nhị phân sau này đều được thể hiện trong cách biểu diễn bằng con trỏ.

5.2.3. Các thao tác trên cây nhị phân

5.2.3.1. Khởi tạo cây nhị phân

Việc khởi tạo cây nhị phân chỉ đơn giản là chúng ta cho con trỏ quản lý địa chỉ nút gốc về con trỏ NULL. Thủ tục khởi tạo như sau:

```
void BTree_Init(Tree *Root)
{
        (*Root) = NULL;
}
```

AUM VIỆT NAM

5.2.3.2. Tạo một nút mới

Việc cài đặt thủ tục này tương tự như cài đặt thủ tục tạo một nút mới cho danh sách liên kết. Thủ tục được cài đặt như sau:

```
Node *CreateNode(ElementType NewData)
{
        Node *p;
        p = (Node*)malloc(sizeof(struct Node));
        if(p!= NULL)
        {
                p -> left = NULL;
                p -> right = NULL;
                p -> Data = NewData;
         }
          return p;
}
```

5.2.3.3. Thêm một nút vào trong cây nhị phân

Việc thêm một nút mới vào vào cây có thể diễn ra ở cây con bên trái hoặc cây con bên phải. Do đó, ta có hai thao tác thêm riêng biệt.

```
//thêm một nút mới vào bên trái của cây.
tạo một nút mới NewNodePtr cần thêm.
if (NewNodePtr == NULL) thì thoát khỏi thác tác chèn
else
{
void Add_Left(ElementType X,Tree Root);
{       if(Root == NULL)
      Tree LPtr;        {
      if(Root == NULL)  Root = NewNodePtr;
      {                 }
         Root = CreateNode(X);  else
      }                 {
      else                LPtr = Root;
      {                   while(LPtr -> left!= NULL)//cây con trái không rỗng
      LPtr = Root;        {
      While (LPtr -> left!= NULL)   LPtr = LPtr -> left;
      {                           } LPtr -> left = NewNodePtr;
      }                   LPtr = LPtr -> left;
}                       }LPtr -> left = CreateNode(X);
                        }
Cài đặt thuật toán:
}
```

AUM VIỆT NAM

Thêm một nút mới vào bên phải nhất của cây nhị phân.

Việc xây dựng thuật toán và cài đặt tương tự như khi thêm một nút mới vào bên trái nhất của cây nhị phân.

5.2.3.4. Duyệt các nút trên cây nhị phân

Phép duyệt cây là phép duyệt các nút trên cây một cách hệ thống sao cho mỗi nút chỉ được thăm một lần. Trong thao tác này, chúng ta tìm cách duyệt qua tất cả các nút trên cây nhị phân để thực hiện một thao tác xử lý nào đó đối với nút (như xem nội dung thành phần dữ liệu). Căn cứ vào thứ tự duyệt nút gốc so với hai nút gốc cây con, thao tác duyệt có thể thực hiện theo một trong ba cách sau:

- Duyệt theo thứ tự trước: Theo cách duyệt này thì nút gốc sẽ được duyệt trước, sau mới đến duyệt 2 nút gốc cây con. Có thể mô tả cách duyệt này bằng thủ tục đệ quy sau:

```
void BTreeTravelling(N)
//duyệt nhánh nhận N làm nút gốc
{
        if (N! = NULL)
        {
                Thủ tục xử lý thông tin của nút N;
                BTreeTravelling(Nút bên trái của N);
                BTreeTravelling(Nút bên phải của N);
        }
}
```

  Trong cách duyệt này căn cứ vào thứ tự duyệt hai cây con mà ta có hai cách duyệt theo thứ tự nút gốc trước:
  - Duyệt nút gốc, duyệt cây con bên trái, duyệt cây con bên phải. Ví dụ, duyệt theo thứ tự này với cây ở hình 5.3 ta có thứ tự duyệt là ABDHEIJCFGK.
  - Duyệt nút gốc, duyệt cây con bên phải, duyệt cây con bên trái. Ví dụ, duyệt theo thứ tự này với cây ở hình 5.3 ta có thứ tự duyệt là ACGKFBEJIDH.
- Duyệt theo thứ tự nút gốc giữa: Theo thứ tự này, chúng ta duyệt một trong hai cây con trước rồi duyệt nút gốc, và sau đó duyệt cây con còn lại. Căn cứ vào thứ tự duyệt hai cây con, ta có hai cách duyệt theo thứ tự nút gốc giữa:
  - Duyệt cây con trái, duyệt nút gốc, duyệt cây con phải. Ví dụ, duyệt theo thứ tự này với cây ở hình 5.3 ta có thứ tự duyệt là HDBEIJAFCGK.
  - Duyệt cây con phải, duyệt nút gốc, duyệt cây con trái. Ví dụ, duyệt theo thứ tự này với cây ở hình 5.3 ta có thứ tự duyệt là KGCFAJEIBDH.
- Duyệt thứ tự nút gốc sau: Theo cách duyệt này thì nút gốc sẽ được duyệt sau cùng so với duyệt hai nút cây con. Căn cứ vào thứ tự duyệt hai cây con mà chúng ta có hai cách duyệt thứ tự nút gốc sau:
  - Duyệt cây con trái, duyệt cây con phải, duyệt nút gốc. Ví dụ, duyệt theo thứ tự này với cây ở hình 5.3 ta có thứ tự duyệt là HDIJEBFKGCA.
  - Duyệt cây con phải, duyệt cây con trái, duyệt nút gốc. Ví dụ, duyệt theo thứ tự này với cây ở hình 5.3 ta có thứ tự duyệt là KGFCJIEHDBA.

Có thể xây dựng các cách duyệt cây nhị phân bằng các thủ tục đệ quy hay không đệ quy. Đây là bài tập cho các học viên tự xây dựng.

AUM VIỆT NAM

5.2.3.5. Hủy một nút trên cây nhị phân

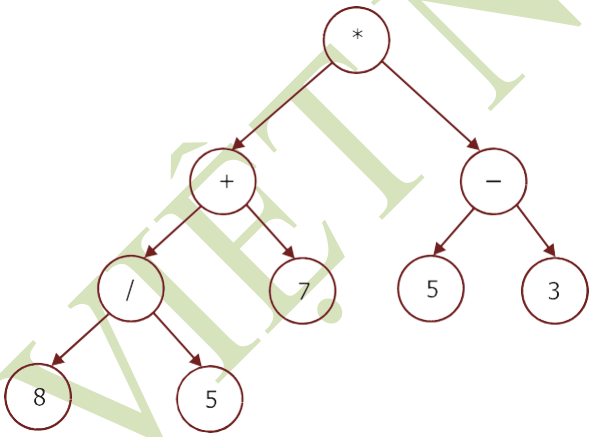
Việc hủy một nút trong cây có thể làm cho cây thành rừng. Vì vậy, khi tiến hành hủy một nút nếu nút đó là lá thì không có điều gì xảy ra. Song, nếu hủy một nút không phải là lá thì chúng ta phải tìm cách chuyển các nút gốc cây con của nút cần hủy thành các nút gốc cây con của các nút khác rồi mới tiến hành hủy nút này.

5.3. Ứng dụng của cây nhị phân

5.3.1. Cây biểu thức

Một ứng dụng về cây nhị phân là cây biểu thức. Cây biểu thức là cây nhị phân gắn nhãn, biểu diễn cấu trúc của một biểu thức (số học hoặc logic), trong đó các nút lá biểu thị các toán hạng (hằng hoặc các biến). Các nút không phải là lá biểu thị các toán tử (các phép toán số học). Mỗi phép toán trong một nút sẽ tác động lên hai biểu thức con nằm ở cây con bên trái và cây con bên phải của nút đó.

Ví dụ 5.4. Xét cây nhị phân biểu diễn biểu thức (8/5 + 7) * (5 – 3)



Hình 5.10. ***Biểu thức dạng cây nhị phân***

Ta có nhận xét rằng, nếu duyệt cây biểu thức ở hình 5.10 theo thứ tự nút gốc trước ta sẽ được * +/857 – 53 đây là dạng **tiền tố** (prefix) (toán tử đứng trước các toán hạng) biểu thức. Cách biểu diễn biểu thức dạng này gọi là **ký pháp Balan**. Nếu duyệt cây biểu thức theo thứ tự nút gốc giữa ta được 8/5 + 7 * 5 – 3, đây là dạng **trung tố** (infix) của biểu thức. Còn nếu duyệt theo thứ tự nút gốc sau ta sẽ được 85/7 + 53 – *, đây là dạng **hậu tố** (postfix) của biểu thức. Trong ký pháp này, toán tử được viết sau hai toán hạng nên người ta còn gọi ký pháp này là ký pháp **nghịch đảo Balan**.

Một vấn đề cần lưu ý khác là máy tính chỉ thực hiện được phép toán với hai toán hạng nên để tính giá trị của một biểu thức phức tạp, máy tính phải chia nhỏ và tính riêng từng biểu thức trung gian sau đó mới lấy giá trị tìm được để tính tiếp. Ví dụ, khi thực hiện phép toán 4 + 2 + 3 máy sẽ tính 4 + 2 trước được 6 sau đó mới đem 6 + 3 chứ không thể thực hiện phép cộng một lúc cả 3 số được. Như vậy, khi lưu trữ biểu thức dưới dạng cây nhị phân thì ta có thể coi mỗi nhánh cây con của cây nhị biểu thức mô tả một biểu thức trung gian mà máy tính cần tính khi xử lí biểu thức lớn. Do vậy, để tính toán biểu thức, máy tính sẽ quan tâm đến việc tính biểu thức ở hai nhánh con trước, rồi mới xét đến toán tử ở nút gốc. Điều đó tương ứng với phép duyệt cây theo

AUM VIỆT NAM

thứ tự nút gốc sau và ký pháp hậu tố. Chúng ta hoàn toàn có thể tính được giá trị biểu thức ở dạng hậu tố bằng cách đọc lần lượt từ trái sang phải và dùng một ngăn xếp (Stack) để lưu các kết quả trung gian theo thuật toán sau:

- **Bước 1:** Khởi tạo một Stack
- **Bước 2:** Đọc lần lượt các phần tử của biểu thức dạng hậu tố từ trái qua phải (phần tử có thể là hạng, biến hay toán tử). Với mỗi phần tử đó, chúng ta kiểm tra:
  - Nếu phần tử này là một toán hạng thì đẩy giá trị của nó vào Stack;
  - Nếu phần là một toán tử ta lấy từ Stack ra hai giá trị, sau đó áp dụng toán tử này vào hai giá trị vừa lấy ra rồi đẩy kết quả tìm được vào Stack (ra hai vào một).
- **Bước 3:** Sau khi kết thúc bước 2 thì trong Stack chỉ còn một giá trị và đó chính là giá trị của biểu thức cần tính.

Ví dụ 5.5.

Dạng hậu tố của biểu thức được biểu diễn ở cây nhị phân hình 5.10 là 85/7 + 53 – *. Việc tính giá trị của biểu thức (8/5 + 7) * (5 – 3) được tính theo dạng hậu tố như sau:

Đọc	Xử lý	Stack
8	Đẩy vào Stack	8
5	Đẩy vào Stack	8,5
/	Lấy 8 và 2 ra khỏi Stack tính được 8/5 = 1.6, đẩy 1.6 vào Stack	1.6
7	Đẩy vào Stack	1.6, 7
+	Lấy 1.6 và 7 ra khỏi Stack tính được 1.6 + 7 = 8.6, đẩy 8.6 vào Stack	8.6
5	Đẩy vào Stack	8.6, 5
3	Đẩy vào Stack	8.6, 5, 3
–	Lấy 3 và 5 ra khỏi Stack tính được 5 – 3 = 2, đẩy 2 vào Stack	8.6, 2
*	Lấy 2 và 8.6 ra khỏi Stack tính được 8.6 * 2 = 17.2, đẩy 17.2 vào Stack	17.2

5.3.2. Cây nhị phân tìm kiếm

Sử dụng cây nhị phân để lưu giữ và tìm kiếm thông tin là một trong những ứng dụng quan trọng nhất của cây nhị phân. Trong mục này chúng ta sẽ xét một lớp cây nhị phân đặc biệt, phục vụ cho việc tìm kiếm thông tin, đó là cây **tìm kiếm nhị phân**. Cây tìm kiếm nhị phân được định nghĩa như sau:

Định nghĩa

Cây tìm kiếm nhị phân là cây nhị phân hoặc trống, hoặc thỏa mãn các điều kiện sau:

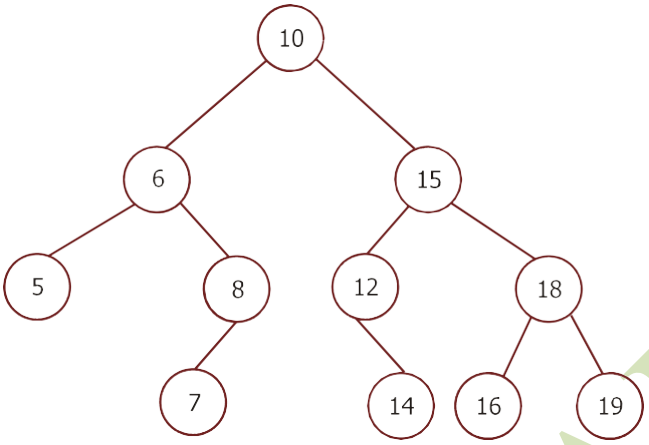
Khóa của các đỉnh thuộc cây con trái nhỏ hơn khóa của gốc.

Khóa của gốc nhỏ hơn khóa của các đỉnh thuộc cây con phải của gốc.

Cây con trái và cây con phải của gốc cũng là cây tìm kiếm nhị phân.

AUM VIỆT NAM

Hình 5.11 biểu diễn một cây tìm kiếm nhị phân, trong đó khoá của các đỉnh là các số nguyên.



Hình 5.11. ***Một cây tìm kiếm nhị phân***

Khi đó qua phép duyệt cây theo thứ tự nút gốc giữa ta sẽ lần lượt đi qua các nút của cây tìm kiếm nhị phân theo thứ tự tăng dần. Điều này dẫn đến ta có một thuật toán sắp xếp khác dựa trên cây nhị phân tìm kiếm.

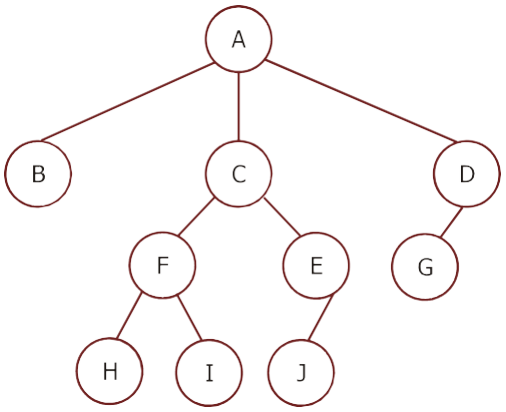
Ví dụ, duyệt cây tìm kiếm nhị phân ở hình 5.11 theo thứ tự nút gốc giữa ta được dãy số sắp xếp theo thứ tự tăng dần: 5, 6, 7, 8, 10, 12, 14, 15, 16, 18, 19.

Ta sẽ nghiên cứu kỹ về cây tìm kiếm nhị phân như cách cài đặt trên máy tính và các thao tác trên cây tìm kiếm nhị phân ở bài sau (bài 6).

5.4. Cây tổng quát

Trong thực tế, có một số ứng dụng đòi hỏi một cấu trúc dữ liệu dạng cây nhưng không bó buộc về số lượng con trên một nút, ví dụ như cấu trúc thư mục trên ổ đĩa hay hệ thống đề mục của một cuốn sách. Cấu trúc cây trong đó, mỗi nút có nhiều hơn hai cây con gọi là cây tổng quát. Trong phần này, chúng ta sẽ trình bày các phương pháp cơ bản cài đặt cây tổng quát.

Ví dụ 5.6. Hình dưới đây mô tả một cây tổng quát



Hình 5.12. ***Mô hình cây tổng quát***

Các phép toán trên cây tổng quát:

Ngoài các phép toán (thao tác) như cây nhị phân, cây tổng quát còn có một số phép

AUM VIỆT NAM

toán cơ bản sau:

- Tìm nút cha của nút bất kỳ trên cây: hàm **PARENT(n)** trả về nút cha của nút n trên cây T, nếu n là nút gốc thì hàm cho giá trị $. Trong cài đặt cụ thể thì $ là một giá trị nào đó do ta chọn. Nó phụ thuộc vào cấu trúc dữ liệu mà ta dùng để cài đặt cây.
- Tìm nút con trái nhất của một nút: hàm **ELDEST_CHILD(n)** cho nút con trái nhất của nút n trên cây T. Nếu n là lá thì hàm cho giá trị $.
- Tìm các nút là anh em ruột của nút n của cây: hàm **NEXT_SIBLING(n)** cho nút anh em ruột phải nút n trên cây T. Nếu n không có anh em ruột phải thì hàm cho giá trị $.

5.4.1. Biểu diễn cây bằng danh sách các con của mỗi đỉnh

Đây là phương pháp thông dụng để biểu diễn cây. Với mỗi đỉnh của cây ta thành lập một danh sách các đỉnh con của nó theo thứ tự từ trái sang phải.

5.4.1.1. Cài đặt bằng mảng

Trong cách cài đặt này, ta sử dụng một mảng để lưu giữ các đỉnh của cây. Mỗi thành phần của mảng là một tế bào chứa thông tin gắn với mỗi đỉnh và danh sách các đỉnh con của nó. Danh sách các đỉnh con của một đỉnh có thể biểu diễn bởi mảng hoặc bởi danh sách liên kết. Tuy nhiên, vì số con của mỗi đỉnh có thể thay đổi nhiều, cho nên ta sử dụng danh sách liên kết. Để tiện cho việc truy cập đến các đỉnh trên cây, ta nên tổ chức lưu trữ các đỉnh này sử dụng cấu trúc mảng. Giả sử ta gán tên cho các nút lần lượt là 1,2,…n. Như vậy, ta mô tả đỉnh của cây là một bản ghi gồm hai trường: trường info chứa thông tin gắn với đỉnh, trường Child là con trỏ tới danh sách các con của đỉnh đó. Giả sử các đỉnh của cây được đánh số từ 1 đến N với cách cài đặt này, ta có thể khai báo cấu trúc dữ liệu biểu diễn cây như sau:

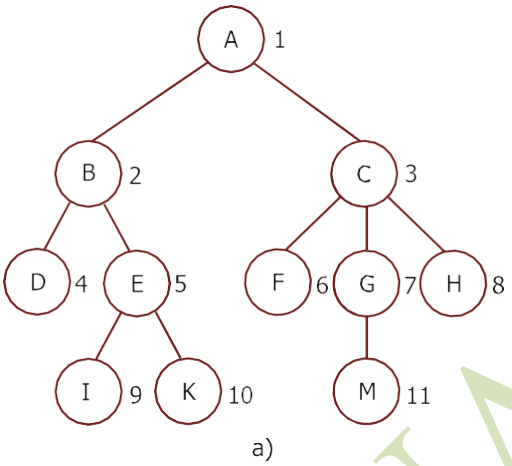
```
#define N…;//N là số lớn nhất các đỉnh mà cây có thể có
typedef <kiểu_dữ_liệu_của_đỉnh> Item;
typedef struct Member
        {
            int id;//id chỉ danh của đỉnh
            Member *Next;//Next:trỏ tới em liền kề
        };
typedef struct Node
        {
            Item infor;
            Member *child;
            //con trỏ trỏ tới con đầu tiên trong danh sách các con
của nó
        };
typedef Node Tree[N];// cây là một mảng
Tree T;
```

Trong khai báo trên, Member biểu diễn các thành phần của danh sách các con, còn Node biểu diễn các đỉnh của cây. Với cách cài đặt này, cấu trúc dữ liệu biểu diễn cây

AUM VIỆT NAM

ở hình 5.13(a) được minh họa theo dạng bảng ở hình 5.13(b).

Hình 5.13. ***Cấu trúc dữ liệu biểu diễn cây***

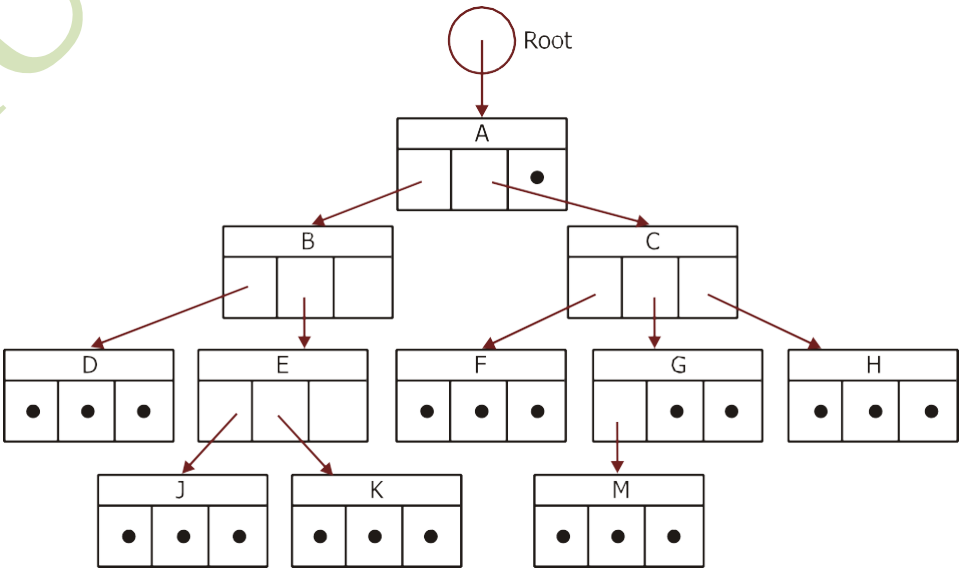


5.4.1.2. Cài đặt bởi con trỏ

Nếu không dùng mảng để lưu giữ các đỉnh của cây, ta có thể sử dụng các con trỏ trỏ tới các đỉnh của cây. Tại mỗi đỉnh, ta sẽ sử dụng một danh sách các con trỏ trỏ tới các con của nó, danh sách này được cài đặt bởi mảng các con trỏ. Một con trỏ Root được sử dụng để trỏ tới gốc của cây. Ta có thể khai báo cấu trúc dữ liệu biểu diễn cây trong cách cài đặt này như sau:

```
#define K…;//K là số tối đa các con của mỗi đỉnh
typedef <kiểu_dữ_liệu_của_đỉnh> Item;
typedef struct Node  //khai báo một đỉnh
        {
           Item info;//lưu thông tin của đỉnh
           Node *childs[K];//chứa các con của đỉnh
        };
typedef struct Node *TreeNode;//khai báo một cây
TreeNode Root; //Root là con trỏ, trỏ tới gốc cây.
```

Với cách cài đặt này, cấu trúc dữ liệu biểu diễn cây bằng con trỏ được minh họa trong hình sau:



AUM VIỆT NAM

5.4.2. Biểu diễn cây bằng con trưởng và em liền kề của mỗi đỉnh

Một phương pháp thông dụng khác để biểu diễn cây là: với mỗi đỉnh của cây ta chỉ ra con trưởng và em liền kề của nó.

5.4.2.1. Cài đặt bởi mảng

Giả sử các đỉnh của cây được đánh số từ 1 đến N. Dùng mảng để lưu giữ các đỉnh của cây, mỗi đỉnh được biểu diễn bởi bản ghi gồm ba trường, ngoài trường infor, các trường EldestChild và NextSibling sẽ lưu con trưởng và em liền kề của mỗi đỉnh. Ta có thể khai báo như sau:

```
typedef <kiểu_dữ_liệu_của_đỉnh> Item;
typedef struct Node
        {
           Item info;//lưu giá trị của phần tử đỉnh
           int EldestChild;//chứa chỉ danh của đỉnh là con trưởng
           int NextSibling;//chứa chỉ danh của đỉnh là em liền kề
        };
Node Tree[N];//cây là một mảng các nút.
```

Hình sau minh hoạ cấu trúc dữ liệu biểu diễn cây trong hình 5.13a.

	Infor	EldestChild	NextSibling
1	A	2	0
2	B	4	3
3	C	6	0
4	D	0	5
5	E	9	0
6	F	0	7
7	G	11	8
8	H	0	0
9	I	0	10
10	K	0	0
11	M	0	0

Hình 5.15. ***Cấu trúc dữ liệu biểu diễn cây***

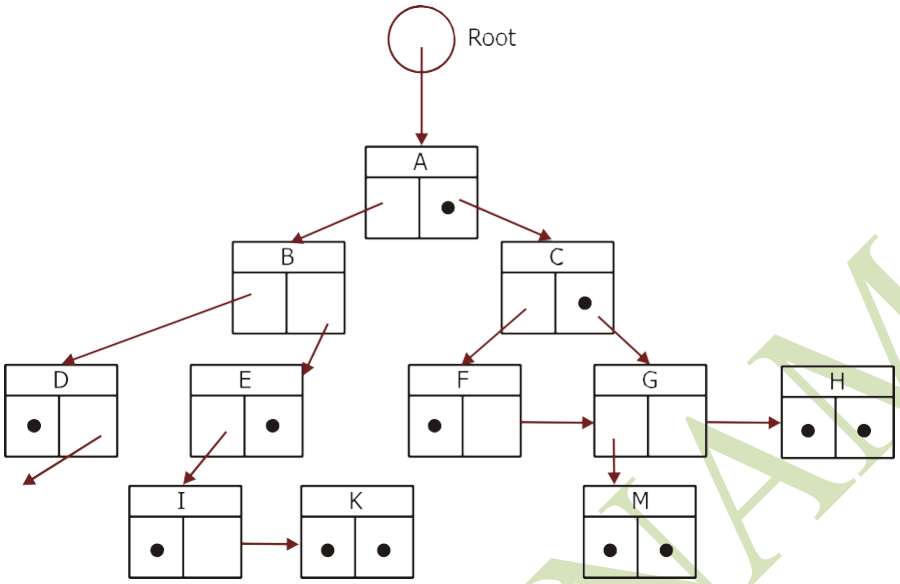
5.4.2.2. Cài đặt bởi con trỏ

Thay cho dùng mảng, ta có thể sử dụng các con trỏ để cài đặt. Khi đó, trong bản ghi Node, các trường EldestChild và NextSibling sẽ là các con trỏ. Cây sẽ được biểu diễn bởi cấu trúc sau.

```
typedef <kiểu_dữ_liệu> Item;
typedef struct Node
    {
        Item infor;
        Node *EldestChild;//trỏ tới gốc cây con cả của đỉnh
        Node *NextSibling;//trỏ tới gốc em liền kề của đỉnh
    };
typedef struct Node *Tree;
Tree Root;
```

AUM VIỆT NAM

Trong cách cài đặt này, cây trong hình 5.13a được biểu diễn bởi cấu trúc dữ liệu trong hình sau:



Hình 5.16. ***Cấu trúc dữ liệu biểu diễn cây***

5.4.3. Biểu diễn cây bởi cha của mỗi đỉnh

Trong một số áp dụng, người ta còn có thể sử dụng cách biểu diễn cây đơn giản sau đây. Giả sử các đỉnh của cây được đánh số từ 1 đến N. Dựa vào tính chất, mỗi đỉnh của cây (trừ gốc) đều có một cha. Ta sẽ dùng một mảng A[1...N] để biểu diễn cây, trong đó A[k] = m nếu đỉnh m là cha của đỉnh k. Trong trường hợp cần quan tâm đến các thông tin gắn với mỗi đỉnh, ta cần phải đưa vào mỗi thành phần của mảng trường infor để mô tả thông tin ở mỗi đỉnh. Cây được biểu diễn bởi cấu trúc sau.

```
#define N …;
typedef <kiểu_dữ_liệu_của_đỉnh> Item;
typedef struct Node
      {
        Item infor;
        int parent;
      } ;
```

Hình 5.17 minh hoạ cấu trúc dữ liệu biểu diễn cây trong hình 5.13 (a).

	infor	**Parent**
1	A	0
2	B	1
3	C	1
4	D	2
5	E	2
6	F	3
7	G	3
8	H	3
9	I	5
10	K	5
11	M	7

Hình 5.17. ***Cấu trúc dữ liệu biểu diễn cây***

AUM VIỆT NAM

5.5. Cây biểu diễn trò chơi

5.5.1. Cây trò chơi

Khi nghiên cứu các trò chơi có hai người tham gia (gọi là **Trắng – Đen**), chẳng hạn các loại cờ (cờ vua, cờ tướng, cờ ca rô...), chúng ta thấy chúng có các đặc điểm sau: hai người chơi thay phiên nhau đưa ra các nước đi tuân theo các luật nào đó. Các luật này là như nhau cho cả hai người. Điển hình là cờ vua. Trong cờ vua, hai người chơi có thể áp dụng các luật đi con tốt, con xe,... để đưa ra nước đi. Luật đi, con tốt Trắng, xe Trắng,... cũng giống luật đi con tốt Đen, xe Đen,... Một đặc điểm khác là hai người chơi đều được biết thông tin đầy đủ về các tình thế trong trò chơi (không như trong chơi bài, người chơi không thể biết các người chơi khác còn những con bài gì). Vấn đề chơi cờ có thể xem như vấn đề tìm kiếm nước đi. Mỗi lần đến lượt mình, người chơi phải tìm trong số rất nhiều nước đi hợp lệ (tuân theo đúng luật đi), một nước đi tốt nhất sao cho qua một dãy nước đi đã thực hiện, anh ta giành phần thắng. Việc tìm kiếm ở đây rất phức tạp vì có đối thủ. Người chơi không biết được đối thủ của mình sẽ đi nước nào trong tương lai.

Vấn đề chơi cờ có thể xem như vấn đề tìm kiếm trong không gian trạng thái. Mỗi trạng thái là một tình thế (sự bố trí các quân của hai bên trên bàn cờ).

- Trạng thái ban đầu là sự sắp xếp các quân cờ của hai bên lúc bắt đầu cuộc chơi.
- Các toán tử là các nước đi hợp lệ.
- Các trạng thái kết thúc là các tình thế mà cuộc chơi dừng, thường được xác định bởi một số điều kiện dừng nào đó.
- Một hàm kết cuộc (payoff function) ứng mỗi trạng thái kết thúc với một giá trị nào đó. Chẳng hạn như cờ vua, mỗi trạng thái kết thúc chỉ có thể là thắng, hoặc thua (đối với Trắng) hoặc hòa. Do đó, ta có thể xác định hàm kết cuộc là hàm nhận giá trị 1 tại các trạng thái kết thúc là thắng (đối với Trắng), –1 tại các trạng thái kết thúc là thua (đối với Trắng) và 0 tại các trạng thái kết thúc hòa. Trong một số trò chơi khác, chẳng hạn trò chơi tính điểm, hàm kết cuộc có thể nhận giá trị nguyên trong khoảng [–k, k] với k là một số nguyên dương nào đó.

Như vậy, vấn đề của người chơi là tìm một dãy nước đi sao cho xen kẽ với các nước đi của đối thủ tạo thành một đường đi từ trạng thái ban đầu tới trạng thái kết thúc là thắng cho mình.

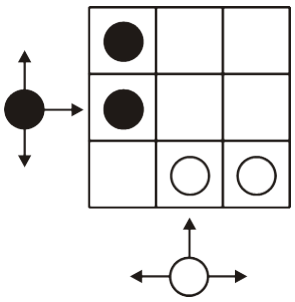
Để thuận lợi cho việc nghiên cứu các chiến lược chọn nước đi, chúng ta biểu diễn không gian trạng thái trên dưới dạng cây gọi là **cây trò chơi**.

Cây trò chơi được xây dựng như sau: Gốc của cây ứng với trạng thái ban đầu. Ta sẽ gọi đỉnh ứng với trạng thái mà Trắng (Đen) đưa ra nước đi là đỉnh Trắng (Đen). Nếu một đỉnh là Trắng (Đen) ứng với trạng thái u, thì các đỉnh con của nó là tất cả các đỉnh biểu diễn trạng thái v. V nhận được từ u do Trắng (Đen) thực hiện nước đi hợp lệ nào đó. Do đó, trên cùng một mức của cây, các đỉnh đều là Trắng hoặc đều là Đen, các lá của cây ứng với các trạng thái kết thúc trò chơi.

Ví dụ 5.7. Xét trò chơi Dodgen (được tạo ra bởi Colin Vout). Có hai quân Trắng và hai quân Đen, ban đầu được xếp vào bàn cờ 3*3 (Hình vẽ 5.18). Quân Đen có thể đi tới ô trống ở bên phải, ở trên hoặc ở dưới. Quân Trắng có thể đi tới trống ở bên trái, bên phải, ở trên. Quân Đen nếu ở cột ngoài cùng bên phải có thể đi ra khỏi bàn cờ, quân Trắng

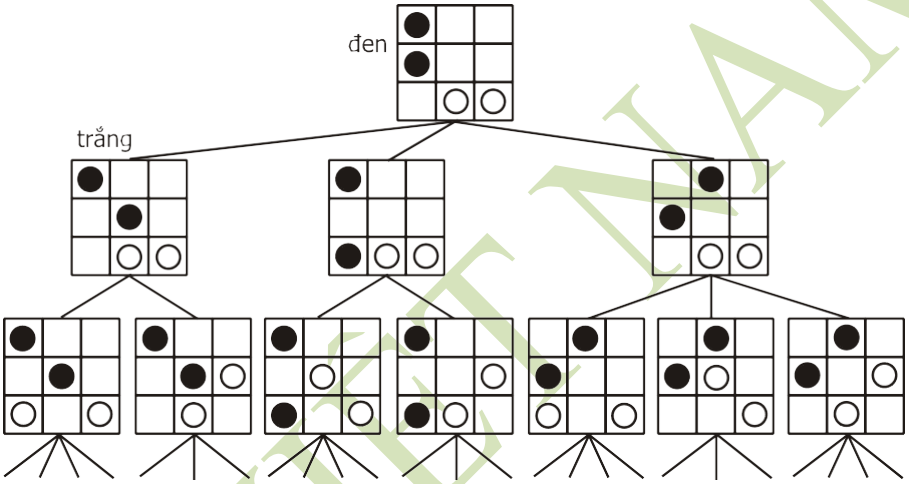
AUM VIỆT NAM

nếu ở hàng trên cùng có thể đi ra khỏi bàn cờ. Ai đưa hai quân của mình ra khỏi bàn cờ trước sẽ thắng, hoặc tạo ra tình thế bắt đối phương không đi được cũng sẽ thắng.



Hình 5.18. ***Trò chơi Dodgen***

Giả sử Đen đi trước, ta có cây trò chơi được biểu diễn như hình 5.19.



Hình 5.19. ***Cây trò chơi Dodgen với đen đi trước***

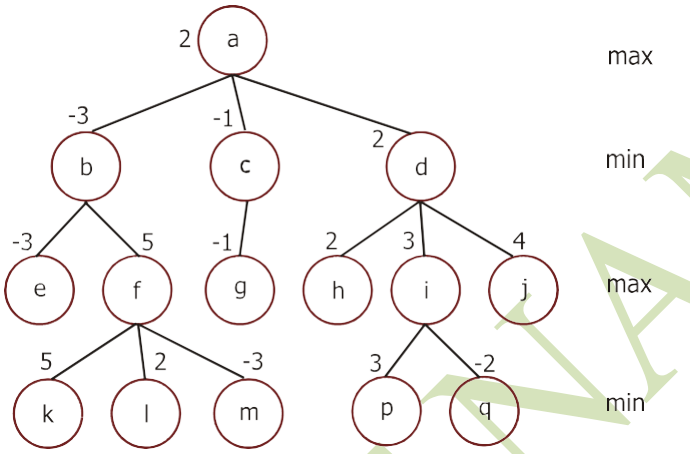
5.5.2. Phương pháp Minimax

Quá trình chơi của các trò chơi hai người (Trắng – Đen) là quá trình Trắng và Đen thay phiên nhau đưa ra quyết định, thực hiện một trong số các nước đi hợp lệ. Trên cây trò chơi, quá trình đó sẽ tạo ra đường đi từ gốc tới lá. Giả sử tới một thời điểm nào đó, đường đi đã dẫn tới đỉnh u. Nếu u là đỉnh Trắng (Đen) thì Trắng (Đen) cần chọn đi tới một trong các đỉnh Đen (Trắng) v là con của u. Tại đỉnh Đen (Trắng) v mà Trắng (Đen) vừa chọn, Đen (Trắng) sẽ phải chọn đi tới một trong các đỉnh Trắng (Đen) u là con của v. Quá trình trên sẽ dừng lại khi đạt tới một đỉnh là lá của cây.

Giả sử Trắng cần tìm nước đi tại đỉnh u. Nước đi tối ưu cho Trắng là nước đi dẫn tới đỉnh con của v là đỉnh tốt nhất (cho Trắng) trong số các đỉnh con của u. Ta cần giả thiết rằng, đến lượt đối thủ chọn nước đi từ v, Đen cũng sẽ chọn nước đi tốt nhất cho anh ta. Như vậy, để chọn nước đi tối ưu cho Trắng tại đỉnh u, ta cần phải xác định giá trị các đỉnh của cây trò chơi gốc u. Giá trị của các đỉnh lá (ứng với các trạng thái kết thúc) là giá trị của **hàm kết cuộc**. Đỉnh có giá trị càng lớn càng tốt cho Trắng, đỉnh có giá trị càng nhỏ càng tốt cho Đen. Để xác định giá trị các đỉnh của cây trò chơi gốc u, ta đi từ mức thấp nhất lên gốc u. Giả sử, v là đỉnh trong của cây và giá trị các đỉnh con của nó đã được xác định, khi đó nếu v là đỉnh Trắng thì giá trị của nó được xác định là giá trị lớn nhất trong các giá trị của các đỉnh con. Còn, nếu v là đỉnh Đen thì giá trị của nó là giá trị nhỏ nhất trong các giá trị của các đỉnh con.

AUM VIỆT NAM

Ví dụ 5.8. Xét cây trò chơi trong hình 5.20. Gốc a là đỉnh Trắng. Giá trị của các đỉnh là số ghi cạnh mỗi đỉnh. Đỉnh i là Trắng, nên giá trị của nó là max(3, –2) = 3, đỉnh d là đỉnh Đen, nên giá trị của nó là min(2, 3, 4) = 2.



Hình 5.20. ***Gán giá trị cho các đỉnh của cây trò chơi***

Việc gán giá trị cho các đỉnh được thực hiện bởi các hàm đệ quy MaxVal và MinVal. Hàm MaxValxác định giá trị cho các đỉnh Trắng. Hàm MinValxác định giá trị cho các đỉnh Đen. Các hàm được xây dựng bằng các thủ tục đệ quy sau:

```
int MaxVal(u) //xác định giá trị cho các đỉnh trắng
{
        if(u là đỉnh kết thúc) MaxVal(u)=f(u);
        else MaxVal(u) = Max{MinVal(v)|v là đỉnh con của u}
};
int MinVal(u) //xác định giá trị cho các đỉnh đen
{
        if(u là đỉnh kết thúc) MinVal(u) = f(u);
        else MinVal(u) = Min{MaxVal(v)|v là đỉnh con của u}
};
```

Trong các hàm đệ quy trên, f(u) là giá trị của hàm kết cuộc tại đỉnh kết thúc u. Sau đây là thủ tục chọn nước đi cho trắng tại đỉnh u. Trong thủ tục Minimax(u,v), v là biến lưu lại trạng thái mà Trắng đã chọn đi tới từ u.

Thủ tục chọn nước đi như trên gọi là **phương pháp Minimax**, bởi vì Trắng đã chọn được nước đi dẫn tới đỉnh con có giá trị là max của các giá trị các đỉnh con, và Đen đáp lại bằng nước đi tới đỉnh có giá trị là min của các giá trị các đỉnh con.

Thuật toán Minimax là thuật toán tìm kiếm theo độ sâu. Ở đây, ta đã cài đặt thuật toán Minimax bởi các hàm đệ quy.

Về mặt lý thuyết, chiến lược Minimax cho phép ta tìm được nước đi tối ưu cho Trắng. Song nó không thực tế. Chúng ta sẽ không có đủ thời gian để tính được nước đi tối ưu, bởi vì thuật toán Minimax đòi hỏi phải xem xét toàn bộ các đỉnh của cây trò chơi. Trong các trò chơi phức tạp, cây trò chơi là cực kỳ lớn. Chẳng hạn, đối với cờ vua, chỉ

AUM VIỆT NAM

tính đến độ sâu 40, cây trò chơi đã có khoảng 10^{120} đỉnh! Nếu cây có độ cao m, và tại mỗi đỉnh có b nước đi thì độ phức tạp về thời gian của thuật toán Minimax là $O(b^m)$.

```
void Minimax(u,v)
{
            val ← -∞;
            for mỗi w là đỉnh con của u
            {
                  if (val <= MinVal(w))
                        {val ← MinVal(w); v ← w}
            }
}
```

Để có thể tìm nhanh nước đi tốt (không phải là tối ưu) thay cho việc sử dụng hàm kết cuộc và xem xét tất cả các khả năng dẫn tới các trạng thái kết thúc, chúng ta sẽ sử dụng hàm đánh giá và chỉ xem xét một bộ phận của cây trò chơi.

Hàm đánh giá

Hàm đánh giá eval ứng với mỗi trạng thái u của trò chơi với một giá trị bằng số eval(u). Giá trị này là sự đánh giá "độ lợi thế" của trạng thái u. Trạng thái u càng thuận lợi cho Trắng thì eval(u) là số dương càng lớn; u càng thuận lợi cho Đen thì eval(u) là số âm càng nhỏ; eval(u) ≈ 0 đối với trạng thái không lợi thế cho ai cả.

Chất lượng của chương trình chơi cờ phụ thuộc rất nhiều vào hàm đánh giá. Nếu hàm đánh giá cho ta sự đánh giá không chính xác về các trạng thái, nó có thể hướng dẫn ta đi tới trạng thái được xem là tốt, nhưng thực tế lại rất bất lợi cho ta. Thiết kế một hàm đánh giá tốt là một việc khó, đòi hỏi ta phải quan tâm đến nhiều nhân tố: các quân còn lại của hai bên, sự bố trí của các quân đó... Ở đây có sự mâu thuẫn giữa độ chính xác của hàm đánh giá và thời gian tính của nó. Hàm đánh giá chính xác đòi hỏi rất nhiều thời gian tính toán, mà người chơi lại bị giới hạn bởi thời gian phải đưa ra nước đi.

Ví dụ 5.9.

Ta đưa ra cách xây dựng hàm đánh giá đơn giản cho cờ vua. Mỗi loại quân được gán một giá trị số phù hợp với "sức mạnh" của nó. Chẳng hạn, mỗi tốt Trắng (Đen) được cho 1 (–1), mã hoặc tượng Trắng (Đen) được cho 3 (–3), xe Trắng (Đen) được cho 5 (–5) và hoàng hậu Trắng (Đen) được cho 9 (–9). Lấy tổng giá trị của tất cả các quân trong một trạng thái, ta sẽ được giá trị đánh giá của trạng thái đó. Hàm đánh giá như thế được gọi là hàm tuyến tính có trọng số, vì nó có thể biểu diễn dưới dạng:

$$s_1w_1 + s_2w_2 + \ldots + s_nw_n$$

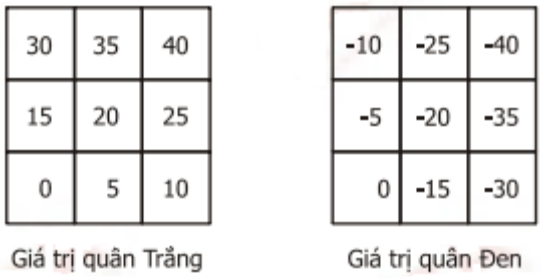
trong đó, w_i là giá trị mỗi loại quân, còn s_i là số quân loại đó. Trong cách đánh giá này, ta đã không tính đến sự bố trí của các quân, các mối tương quan giữa chúng.

Ví dụ 5.10.

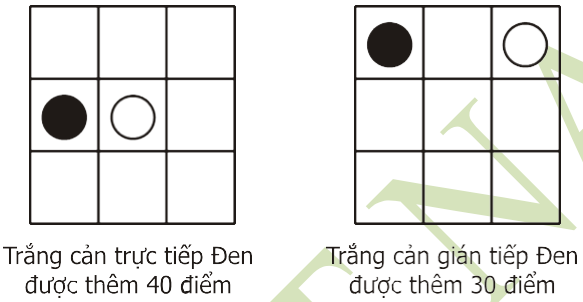
Bây giờ ta đưa ra một cách đánh giá các trạng thái trong trò chơi Dodgen. Mỗi quân Trắng ở một vị trí trên bàn cờ được cho một giá trị tương ứng trong bảng bên trái hình 5.21. Còn mỗi quân Đen ở một vị trí sẽ được cho một giá trị tương ứng trong bảng bên phải hình 5.21:

AUM VIỆT NAM

Hình 5.21. ***Đánh giá các quân trong trò chơi Dodgen***

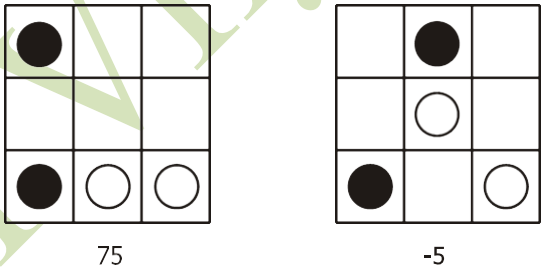


Ngoài ra, nếu quân Trắng cản trực tiếp một quân Đen, nó được thêm 40 điểm, nếu cản gián tiếp nó được thêm 30 điểm (Xem hình 5.22). Tương tự, nếu quân Đen cản trực tiếp quân Trắng nó được thêm –40 điểm, còn cản gián tiếp nó được thêm –30 điểm.



Hình 5.22. ***Đánh giá tương quan giữa quân Trắng và Đen***

Áp dụng các quy tắc trên, ta tính được giá trị của trạng thái ở bên trái hình 5.23 là 75, giá trị của trạng thái bên phải hình vẽ là –5.



Hình 5.23. ***Giá trị của một số trạng thái trong trò chơi Dodgen***

Trong cách đánh giá trên, ta đã xét đến vị trí của các quân và mối tương quan giữa các quân. Một cách đơn giản để hạn chế không gian tìm kiếm là, khi cần xác định nước đi cho Trắng tại u, ta chỉ xem xét cây trò chơi gốc u tới độ cao h nào đó. Áp dụng thủ tục Minimax cho cây trò chơi gốc u, độ cao h và sử dụng giá trị của hàm đánh giá cho các lá của cây đó, chúng ta sẽ tìm được nước đi tốt cho Trắng tại u.

Phương pháp minimax với cơ chế cắt tỉa alpha – beta

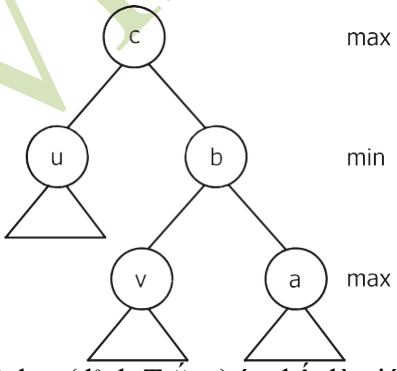
Trong chiến lược tìm kiếm Minimax, để tìm kiếm nước đi tốt cho Trắng tại trạng thái u, cho dù ta hạn chế không gian tìm kiếm trong phạm vi cây trò chơi gốc u với độ cao h, thì số đỉnh của cây trò chơi này cũng còn rất lớn với $h \geq 3$. Chẳng hạn, trong cờ vua, nhân tố nhánh trong cây trò chơi trung bình khoảng 35, thời gian đòi hỏi phải đưa ra nước đi là 150 giây, với thời gian này trên máy tính thông thường chương trình của bạn chỉ có thể xem xét các đỉnh trong độ sâu 3 hoặc 4. Một người chơi cờ trình độ trung bình cũng có thể tính trước được 5, 6 nước hoặc hơn nữa, và do đó chương trình của bạn mới đạt trình độ người mới tập chơi!

AUM VIỆT NAM

```
int MaxVal(u, α, β);
{
        if (u là lá của cây hạn chế hoặc u là đỉnh kết thúc)
                MaxVal ← eval(u)
        else
        {
                for (mỗi đỉnh v là con của u)
                {
                        α ← max[α, MinVal(v, α, β)];
                       // Cắt bỏ các cây con từ các đỉnh v còn lại
                        if (α ≥ β) exit;
                 }
        }
        MaxVal ← α;
}
```

Khi đánh giá đỉnh u tới độ sâu h, một thuật toán Minimax đòi hỏi ta phải đánh giá tất cả các đỉnh của cây gốc u tới độ sâu h. Song ta có thể giảm bớt số đỉnh cần phải đánh giá mà vẫn không ảnh hưởng gì đến sự đánh giá đỉnh u. Phương pháp cắt cụt alpha–beta cho phép ta cắt bỏ các nhánh không cần thiết cho sự đánh giá đỉnh u.

Tư tưởng của kỹ thuật cắt cụt alpha – beta là như sau: Nhớ lại rằng, chiến lược tìm kiếm Minimax là chiến lược tìm kiếm theo độ sâu. Giả sử trong quá trình tìm kiếm, ta đi xuống đỉnh a là đỉnh Trắng. Đỉnh a có người anh em v đã được đánh giá. Giả sử cha của đỉnh a là b và b có người anh em u dã được đánh giá. Giả sử cha của b là c (Xem hình



5.24). Khi đó ta có giá trị đỉnh c (đỉnh Trắng) ít nhất là giá trị của u, giá trị của đỉnh b (đỉnh Đen) nhiều nhất là giá trị v. Do đó, nếu eval(u) > eval(v), ta không cần đi xuống để đánh giá đỉnh a nữa mà vẫn không ảnh hưởng gì dến đánh giá đỉnh c. Nói cách khác, ta có thể cắt bỏ cây con gốc a. Lập luận tương tự cho trường hợp a là đỉnh Đen, trong trường hợp này nếu eval(u)< eval(v)ta cũng có thể cắt bỏ cây con gốc a. Để cài đặt kỹ thuật cắt cụt alpha – beta, đối với các đỉnh nằm trên đường đi từ gốc tới đỉnh hiện thời, ta sử dụng tham số α để ghi lại giá trị lớn nhất trong các giá trị của các đỉnh con đã đánh giá của một đỉnh Trắng, còn tham số ghi lại giá trị nhỏ nhất trong các đỉnh con đã đánh giá của một đỉnh Đen. Giá trị của α và β sẽ được cập nhật trong quá trình tìm kiếm. α và β được sử dụng như các biến địa phương trong các hàm MaxVal(u, α, β) (hàm xác định giá trị của đỉnh Trắng u) và Minval(u, α, β) (hàm xác định giá trị của đỉnh Đen u).

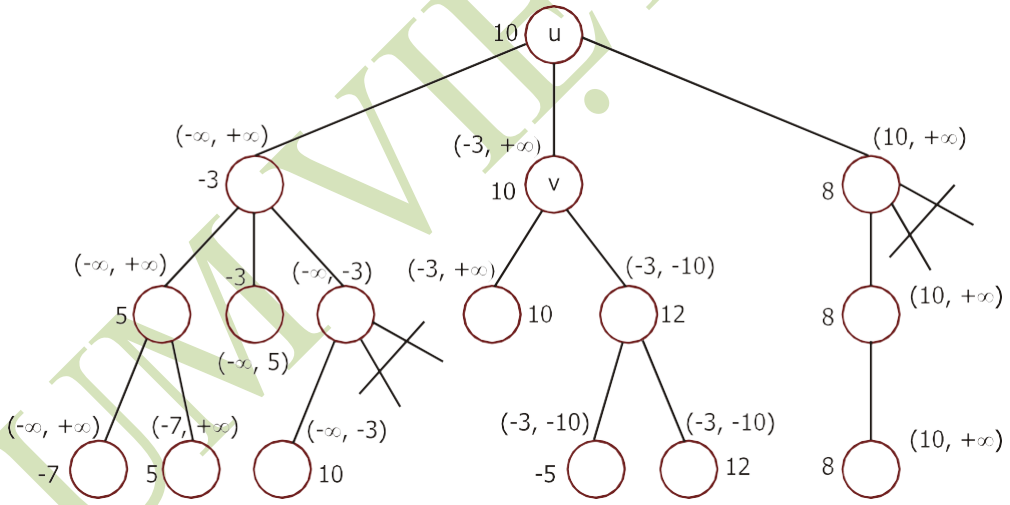
AUM VIỆT NAM

Thuật toán tìm nước đi cho Trắng sử dụng kỹ thuật cắt cụt alpha – beta, được cài đặt bởi thủ tục Alpha_beta(u,v), trong đó, v là tham biến ghi lại đỉnh mà Trắng cần đi tới từ u.

```
void Alpha_beta(u,v)
{
        α ← −∞;
        β ← ∞;
        for (mỗi đỉnh w là con của u)
                if (α ≤ MinVal(w, α, β))
                {
                        α ← MinVal(w, α, β);
                        v ← w;
                }
}
```

Ví dụ 5.11.

Xét cây trò chơi gốc u (đỉnh Trắng) giới hạn bởi độ cao h = 3 (hình 5.25). Số ghi cạnh các lá là giá trị của hàm đánh giá. Áp dụng chiến lược Minimax và kỹ thuật cắt cụt, ta xác định được nước đi tốt nhất cho Trắng tại u, đó là nước đi dẫn tới đỉnh v có giá trị 10. Cạnh mỗi đỉnh, ta cũng cho giá trị của cặp tham số (α, β). Khi gọi các hàm MaxVal và MinVal để xác định giá trị của đỉnh đó, các nhánh bị cắt bỏ được chỉ ra trong hình:



Hình 5.25. ***Xác định giá trị các đỉnh bằng kỹ thuật cắt cụt***

5.5.3. Trò chơi Tic – Tac - Toe

Tic - tac - toe là một trò chơi phổ biến dùng cách viết trên bàn cờ giấy có chín ô, 3x3 cho hai người chơi. Một trong hai đối thủ sẽ đi trước, đánh X (hoặc O) vào một ô trống bất kỳ trên bàn cờ. Hai người thay phiên nhau đánh vào các ô trống cho đến khi một người tạo được một dãy ba ký hiệu của mình, ngang dọc hay chéo đều được thì thắng. Nếu hết 9 ô cờ mà không người chơi nào có được 3 quân cờ của mình nằm trên một đường thẳng thì ván đó hòa.

Người Việt chúng ta vẫn thường chơi trò tương tự, gọi là **Cờ ca – rô**, bàn cờ không giới hạn trong 9 ô, có thể vẽ thêm ô, để kéo rộng ra cho đến khi người nào đạt được một dãy 5 thì thắng cuộc.

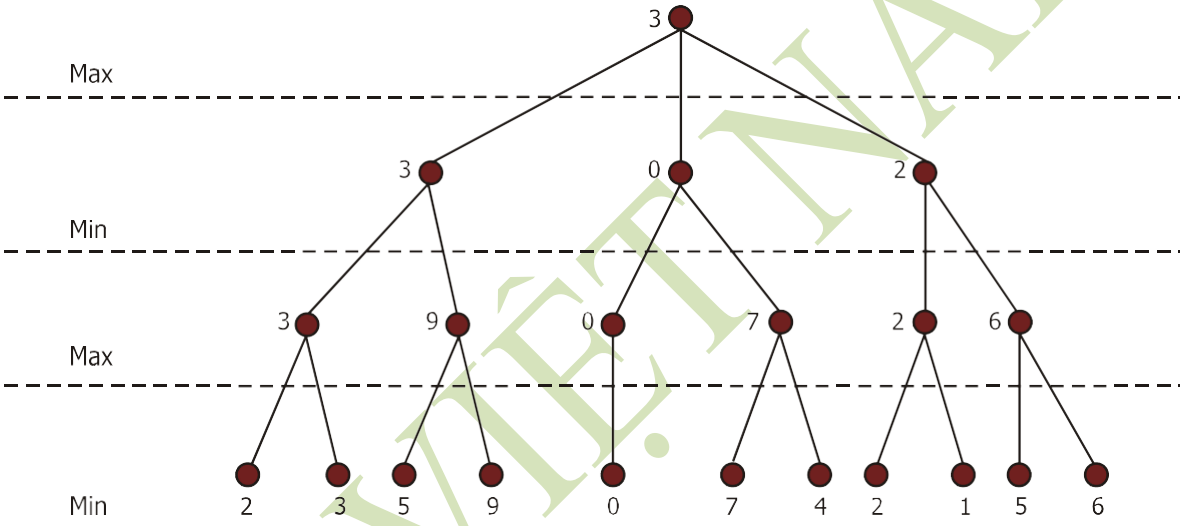
AUM VIỆT NAM

Như vậy, **mục tiêu của trò chơi** là đến lượt, mỗi người chơi cố gắng tạo ra 3 quân cờ trên một đường thẳng để là người chiến thắng hoặc ngăn cản người còn lại tạo ra 3 quân cờ trên một đường thẳng.



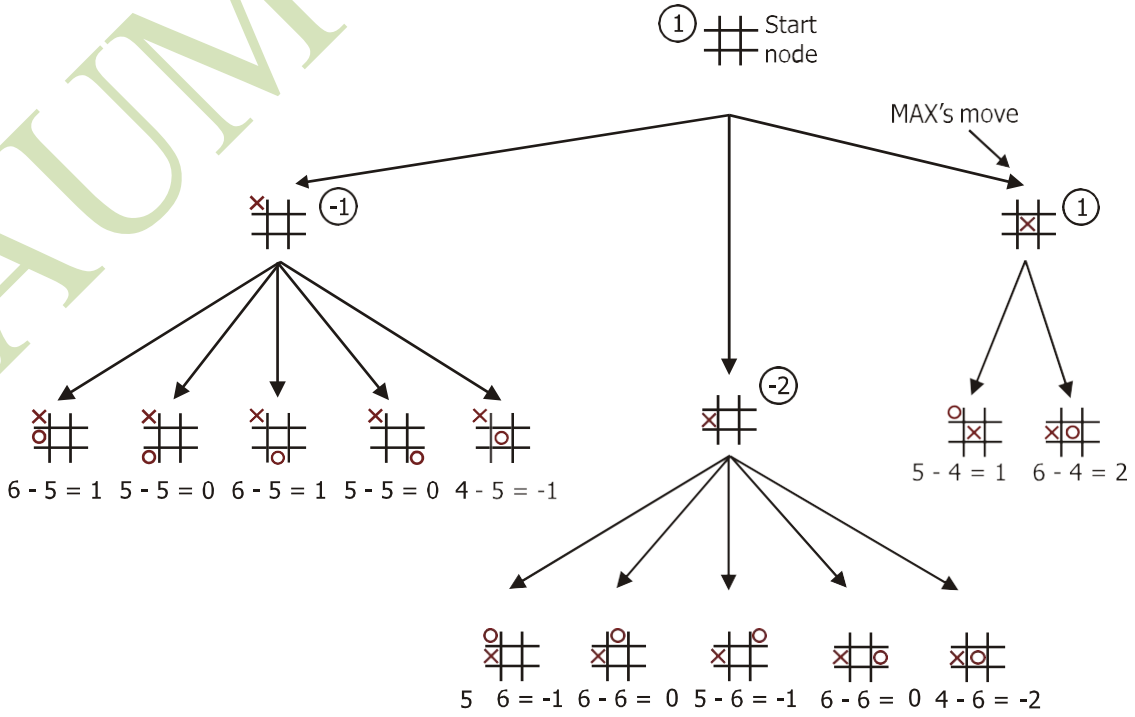
Hình 5.26: *Trò chơi Tic-tac-toe với phần thắng thuộc về người O*

Để giải quyết bài toán này chúng ta sử dụng giải thuật minimax – cơ chế cắt tỉa alpha beta để tìm nước đi tốt nhất cho từng người chơi. Giả sử hai người chơi gọi là MIN va MAX. MAX đại diện cho người chơi quyết dành thắng lợi hay cố gắng tối ưu hóa ưu thế của mình. Ngược lại MIN là người chơi cố tối thiểu hóa điểm số của MAX. Khi áp dụng thủ thuật minimax, chúng ta đánh dấu luân phiên từng mức trong không gian tìm kiếm phù hợp với đối thủ có nước đi ở mức đó. Kết quả của việc áp dụng Minimax vào cho chơi Tic – tac – toe được thể hiện như hình 5.27 .



Hình 5.27. ***Cây thể hiện áp dụng giải thuật minimax cho trò chơi Tic – tac – toe***

Áp dụng vào trò chơi



AUM VIỆT NAM

TÓM LƯỢC CUỐI BÀI

Trong bài này chúng ta tìm hiểu một dạng cấu trúc dữ liệu trừu tượng, đó chính là cây. Tìm hiểu sâu về một dạng đặc biệt của cây đó là cây nhị phân. Các bạn cần lưu ý một số vấn đề sau:

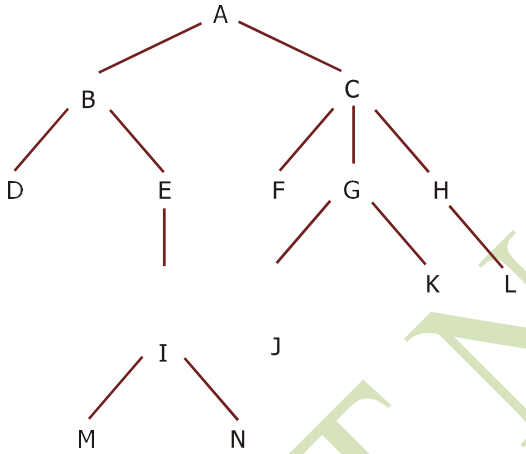
- Trình bày đúng khái niệm về cây, cây con, độ cao mức của 1 đỉnh…
- Trình bày đúng khái niệm cây nhị phân, cách cài đặt cây nhị phân, các thao tác trên cây nhị phân.
- Trình bày được các ứng dụng cây nhị phân trong giải quyết các bài toán.

AUM VIỆT NAM

BÀI TẬP

1. Trình bày khái niệm, đặc điểm và cấu trúc dữ liệu của các cây nhị phân và cây tổng quát? So sánh với danh sách liên kết?

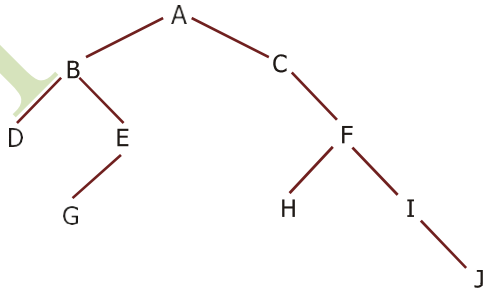
2. Trình bày các biểu thức duyệt tiền tự, trung tự, hậu tự của cây sau:



3. Duyệt cây theo mức là duyệt bắt đầu từ gốc, rồi duyệt các nút nằm trên mức 1 theo thứ tự từ trái sang phải, rồi đến các nút nằm trên mức 2 theo thứ tự từ trái sang phải... và cứ như vậy. Hãy liệt kê các nút theo thứ tự duyệt theo mức của cây trong bài 2.

4. Vẽ cây nhị phân biểu diễn cho biểu thức ((a + b) + c * (d + e) + f) * (g + h). Trình bày biểu thức tiền tố và hậu tố của biểu thức đã cho.

5. Cho cây nhị phân



a) Hãy trình bày kết quả của các phép duyệt: tiền tự (node – left – right), trung tự (left – node – right), hậu tự (left – right – node).

b) Minh hoạ sự lưu trữ kế tiếp các nút cây này trong mảng.

AUM VIỆT NAM